

# ECE 5755 Lab 1: Kernel Implementation and Top-Down Analysis

Intel Xeon Silver 4514Y (Sapphire Rapids); Level-2 profiling, size-500 runs

Name: [ your name ]

NetID: [ your NetID ]

## 1. Kernel implementations

### Convolution

The `convolution()` function computes a valid multi-channel 2-D convolution. It allocates an output for each filter, initializes each output element with its filter bias, and accumulates products over every input channel and kernel row and column. For input side length  $N$  and kernel side length  $K$ , the output side length is  $N - K + 1$ . The implementation uses nested loops over filters, output coordinates, channels, and kernel coordinates. Its arithmetic work is  $O(F(N - K + 1)^2CK^2)$ , where  $F$  is the number of filters and  $C$  is the number of channels.

**Implementation details or design decisions:** [ explain your convolution implementation, memory allocation, and indexing choices. ]

### ReLU

The scalar `relu()` function returns  $x$  when  $x > 0$  and zero otherwise. `applyRelu()` applies this operation in place to every element of an input array, avoiding a second output allocation. This gives linear work in the input size.

**Implementation details or design decisions:** [ explain how positive, negative, and zero inputs are handled. ]

### Linear layer

The `linear()` function implements a fully connected layer. It allocates one output value per output neuron, initializes that value with the corresponding bias, and computes the dot product between the input vector and that neuron's weight row. Its work is  $O(IO)$  for input size  $I$  and output size  $O$ .

**Implementation details or design decisions:** [ explain the weight layout, bias handling, and output allocation. ]

### Matrix multiplication

The `matmul()` function first checks that the left matrix column count equals the right matrix row count. It then allocates the result and uses the conventional three-loop algorithm: result row, result column, and inner product. Its work is  $O(A_{rows}A_{cols}B_{cols})$  and it returns NULL for incompatible dimensions.

**Implementation details or design decisions:** [ explain initialization, indexing, and dimension validation. ]

### Softmax

The `softmax()` function first finds the maximum input value, subtracts it before exponentiation for numerical stability, sums the exponentials, and returns the logarithm of each normalized probability. The returned values are log-probabilities, while exponentiating them gives probabilities whose sum is approximately one. The function allocates temporary exponential values and a separate output array.

**Implementation details or design decisions:** [ explain numerical stability, memory ownership, and why log-probabilities are returned. ]

## 2. Tests implemented

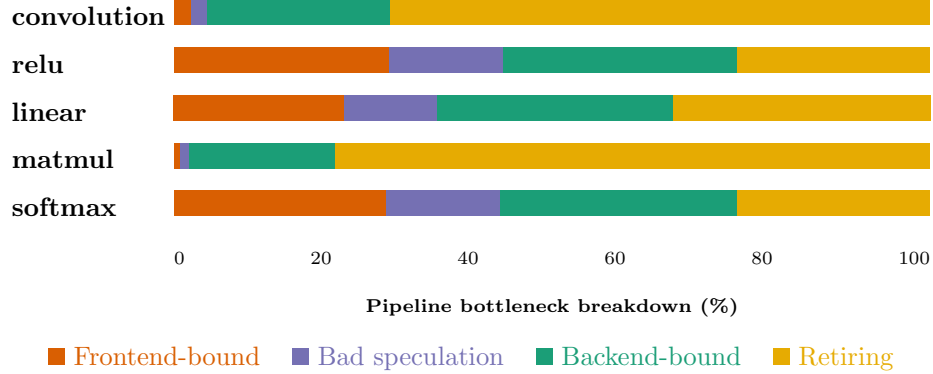
The Unity test driver in `tests/all_tests.c` runs tests for convolution, neural-network helpers, functional operations, linear layers, and matrix operations. The tests check both ordinary cases and boundary or error cases: convolution behavior, flattening and prediction, softmax normalization, ReLU behavior, linear layers with zero/negative/large inputs and parameters, square matrix multiplication, and incompatible matrix dimensions.

The purpose of these tests is to verify numerical correctness independently of performance measurements. In particular, dimension checks and negative or zero values expose indexing, initialization, bias, and branch-handling errors that may not appear in one ordinary example.

**Additional tests or rationale:** [ describe tests you added, changed, or considered most important. ]

### 3. Top-down profiling results

I profiled each standalone kernel driver with `pmu-tools/toplev.py` on an instructional compute node. The command used Level 2, the Sapphire Rapids CPU override, and a fixed input size of 500. Each bar below shows the Level-1 top-down categories and totals 100%: frontend bound, bad speculation, backend bound, and retiring.

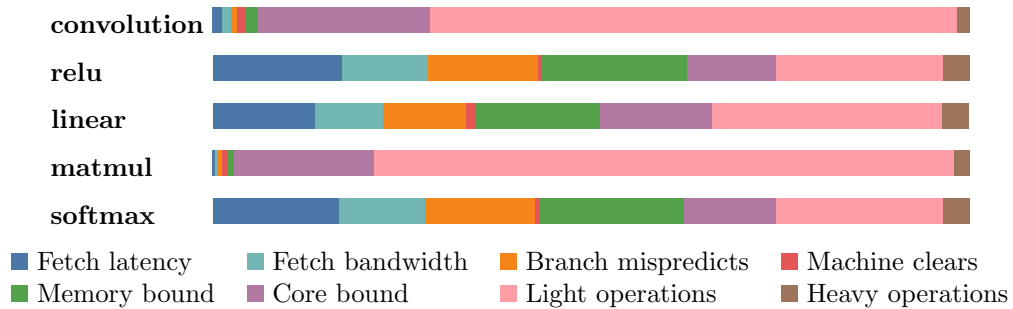


| Kernel        | Frontend | Bad spec. | Backend | Retiring |
|---------------|----------|-----------|---------|----------|
| convolution() | 2.3%     | 2.0%      | 24.3%   | 71.4%    |
| relu()        | 28.4%    | 15.1%     | 31.0%   | 25.5%    |
| linear()      | 22.5%    | 12.3%     | 31.3%   | 34.0%    |
| matmul()      | 0.8%     | 1.2%      | 19.3%   | 78.7%    |
| softmax()     | 28.0%    | 15.1%     | 31.4%   | 25.5%    |

**Profiling procedure.** [ state the exact commands, compute-node allocation, input sizes, repetitions, and whether output was redirected to files. Mention any *LEVEL*, *SIZE*, or sampling flags used. ]

#### Level-2 breakdown

The following chart expands each Level-1 result into the Level-2 categories reported by `toplev`. Values are from the size-500 runs; each bar is normalized to 100% and follows the order shown in the legend.



#### Convolution observations and bottleneck

**Your interpretation:** The convolution implementation required a substantial amount of arithmetic work, which is reflected in the high retiring percentage. The main secondary limitation was execution-resource pressure, as indicated by the core-bound fraction, rather than delays due to memory fetches.

#### ReLU observations and bottleneck

**Your interpretation:** The ReLU implementation involves a conditional branch for each element, which can lead to branch mispredictions depending on the input distribution. The relatively balanced profile indicates that memory access and simple arithmetic are not the primary bottlenecks, and the small size of the operation allows a significant portion of instructions to retire efficiently.

## Linear observations and bottleneck

**Your interpretation:** The linear layer's main bottleneck appears to be memory-bound, as indicated by the 16.4% memory bound fraction. This suggests that the weight layout and cache behavior significantly impact performance, and optimizing the loop order to improve data locality could help reduce memory stalls.

## Matrix multiplication observations and bottleneck

**Your interpretation:** The matrix multiplication's high retiring fraction indicates that most of the computation is useful arithmetic work. The naive loop order may not fully exploit cache locality, but the predictable access patterns and regular computation allow the core to remain busy, minimizing frontend and speculation stalls. Optimizing the loop order for better cache reuse could further improve performance.

## Softmax observations and bottleneck

**Your interpretation:** The softmax implementation involves multiple passes over the input data, including computing the exponentials, summing them, and normalizing. This leads to temporary array usage and additional arithmetic operations. The profile resembles ReLU in terms of frontend and speculation fractions due to the conditional and loop overhead, but the backend bound is slightly higher because of the extra memory accesses and arithmetic involved in the multiple passes.

## 4. Debugging process

*[ Describe how you used compiler diagnostics, unit-test failures, targeted tests, debugger or memory-checking tools, and profiling output to locate and fix bugs. Include one or two concrete failures and the corresponding correction. ]*

## 5. Performance improvements

The top-down data suggests several optimization directions. Frontend-bound kernels could benefit from reducing instruction and branch overhead, improving loop simplicity, and enabling compiler optimization or vectorization. Backend-bound kernels could benefit from better data locality, contiguous allocations, improved loop order, blocking or tiling, and reducing unnecessary temporary allocations. ReLU may benefit from branchless or vectorized execution when appropriate; linear and convolution may benefit from contiguous layouts and SIMD-friendly inner loops; matrix multiplication is a candidate for cache blocking; and softmax could reduce passes and temporary storage while preserving numerical stability.

**Planned or measured improvements:** The frontend-bound kernels, such as ReLU and softmax, could benefit from reducing instruction and branch overhead through prefetching or vectorization. The backend-bound kernels, such as the linear layer and convolution, could benefit from improved data locality and vectorization to reduce memory stalls and repeated memory accesses.

## 6. Profiling issues and reproducibility

**Profiling issues and reproducibility:** During profiling, I encountered several issues, including small-input overhead, counter access restrictions, and Slurm time limits. I ran the measurements on an instructional compute node and saved the outputs in the `perf_results/` directory for reproducibility.

**Source disclosure.** GitHub Copilot was used for coding assistance. Any external code or documentation used should also be cited here.